

[기업연계] 언리얼 엔진 게임 개발자 부트캠프 8기

Technical documentation

PROJECT

BLACKOUT

언리얼 엔진을 활용한 멀티플레이 게임
3인칭 소울라이크 PvE 협동 보스 러시

김민영

zxc9876zxc@gmail.com



목차

01	프로젝트 개요	게임 소개 (04) · 개발 환경 (05) · 게임 흐름도 (06 ~ 07)
02	담당 역할	팀 내 담당 업무 및 GitHub 기여 내역 (09)
03	핵심 기능 구현	카메라-총구 시차 보정 (11 ~ 12) · 표면별 GCN 선택 로직 (13 ~ 14) · 서버 권위 및 네트워크 지연 보정 (15 ~ 17)
04	프로젝트 회고	개선점 & 느낀 점 (19)

01

프로젝트 개요

게임 소개 · 개발 환경 · 게임 흐름도

게임 소개



게임명

Project Blackout

장르

TPS 소울라이크 협동 PvE (4인 협동 보스 레이드)

플랫폼

Client: PC (Windows) / Server: AWS (Linux)

개발 기간

2026.04.25 ~ 2026.06.19 (8주)

인원

4인 (프로그래머)

프로젝트 링크

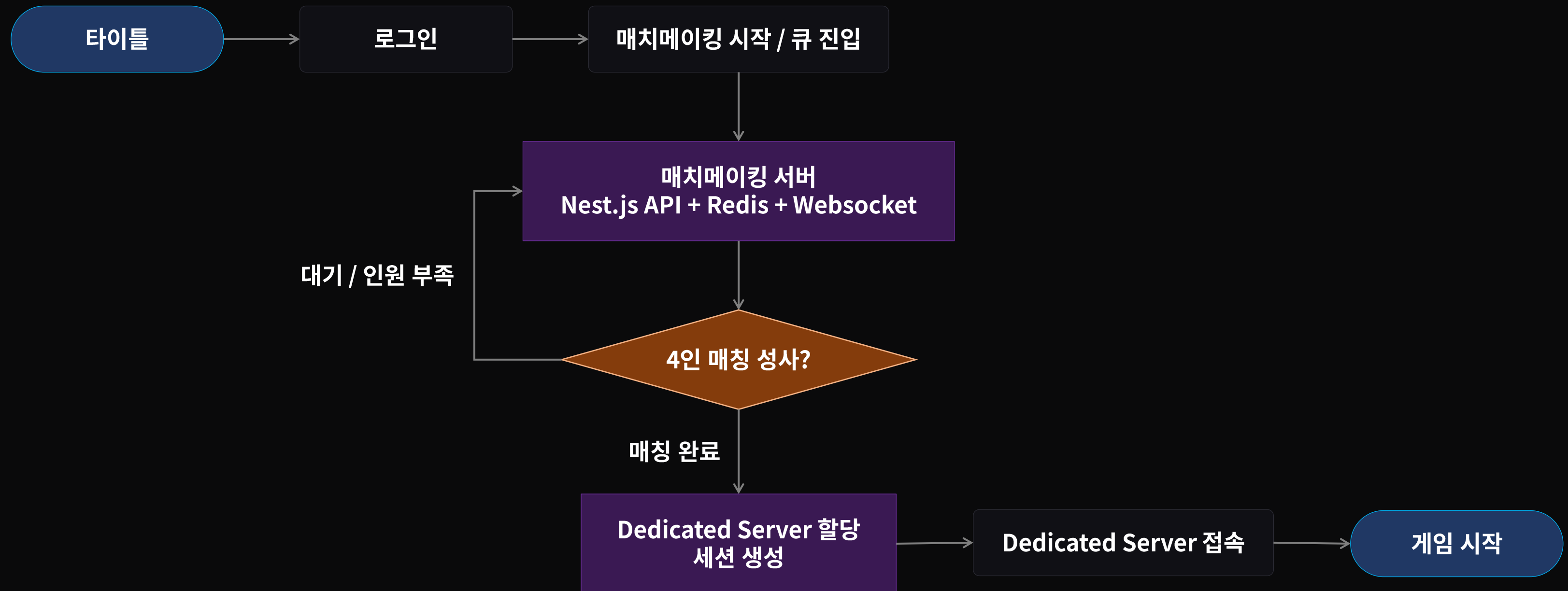


개발 환경

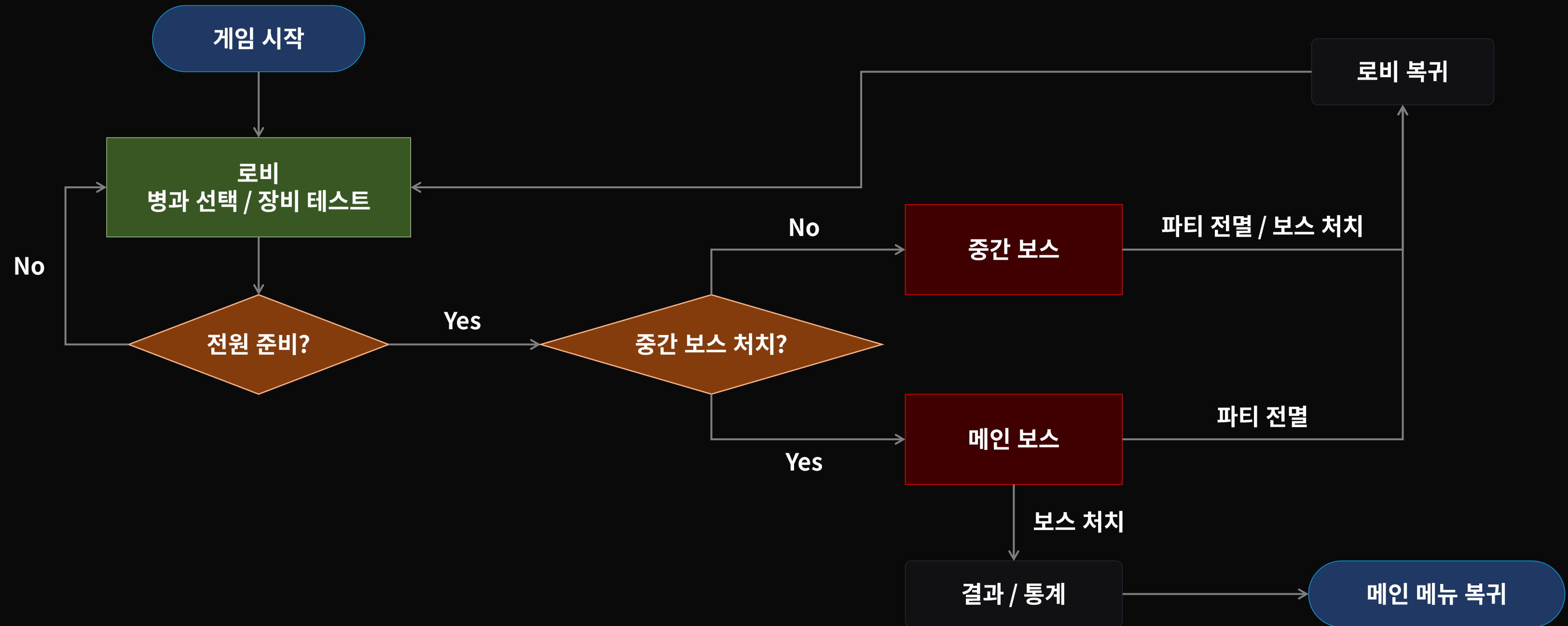


게임 엔진	Unreal Engine 5.7.4 — 바이너리 빌드 버전 고정
IDE	Rider
형상 관리	코드 GitHub / 바이너리(.uasset) Google Drive
네트워크	데디케이티드 서버 전용 (AWS)
매치메이킹 서버	Nest.js + Redis + WebSocket
협업	Discord · Notion (Epic/Task)

게임 흐름도 - 아웃게임



게임 흐름도 - 인게임





02

담당 역할

팀 내 담당 업무 및 GitHub 기여 내역

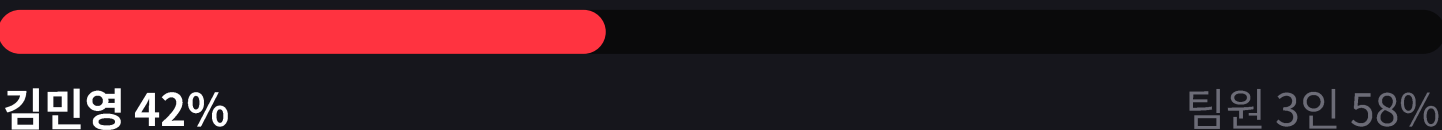
담당 역할 및 기여

개발 컨벤션 확립 · Epic/Task 분배 · 일정 관리를 맡으며 주요 시스템을 직접 설계·구현

CONTRIBUTION

440 commits

Git 유효 commit 기준 (merge 제외)
팀 내 **최다 기여** · 유효 commit 1,037건 중 약 42%



- GitHub 기여 기록으로 검증

담당 분야

- Core
- GAS
- Combat
- GCN
- UI/UX

담당 역할

팀장

담당 업무

- 개발 컨벤션 확립 및 AI 에이전트 지침서 작성
- 프로젝트 내 주요 시스템의 설계 · 구현
- 팀원 별 Epic/Task 분배 · Notion 기반 일정 관리

03

핵심 기능 구현

카메라-총구 시차 보정 · GCN 실행 로직 · 서버 권위 및 네트워크 지연 보정

카메라-총구 시차 보정

카메라-총구 시차(Parallax) 보정 — Aim Offset을 활용한 캐릭터 상체 정렬

문제 : FPS/TPS 게임에서는 **카메라의 위치와 총구 위치가 달라** 카메라-총구 간의 **시차 문제 발생**

- FPS와 TPS 등의 슈팅 게임의 경우 카메라의 위치와 총구의 위치가 다름
- 총알이 정직하게 총구에서 나가면? → 총알이 **조준점에 안 맞음**
- 총알이 카메라에서 나가면? → **카메라만 내놓고 사격** 가능
- 돈이 아주 많은 AAA급이 아닌 대부분의 게임은 **카메라에서 총알이 나가거나** 조준점을 향해 **총알이 휘어지게 만들어** 문제 회피

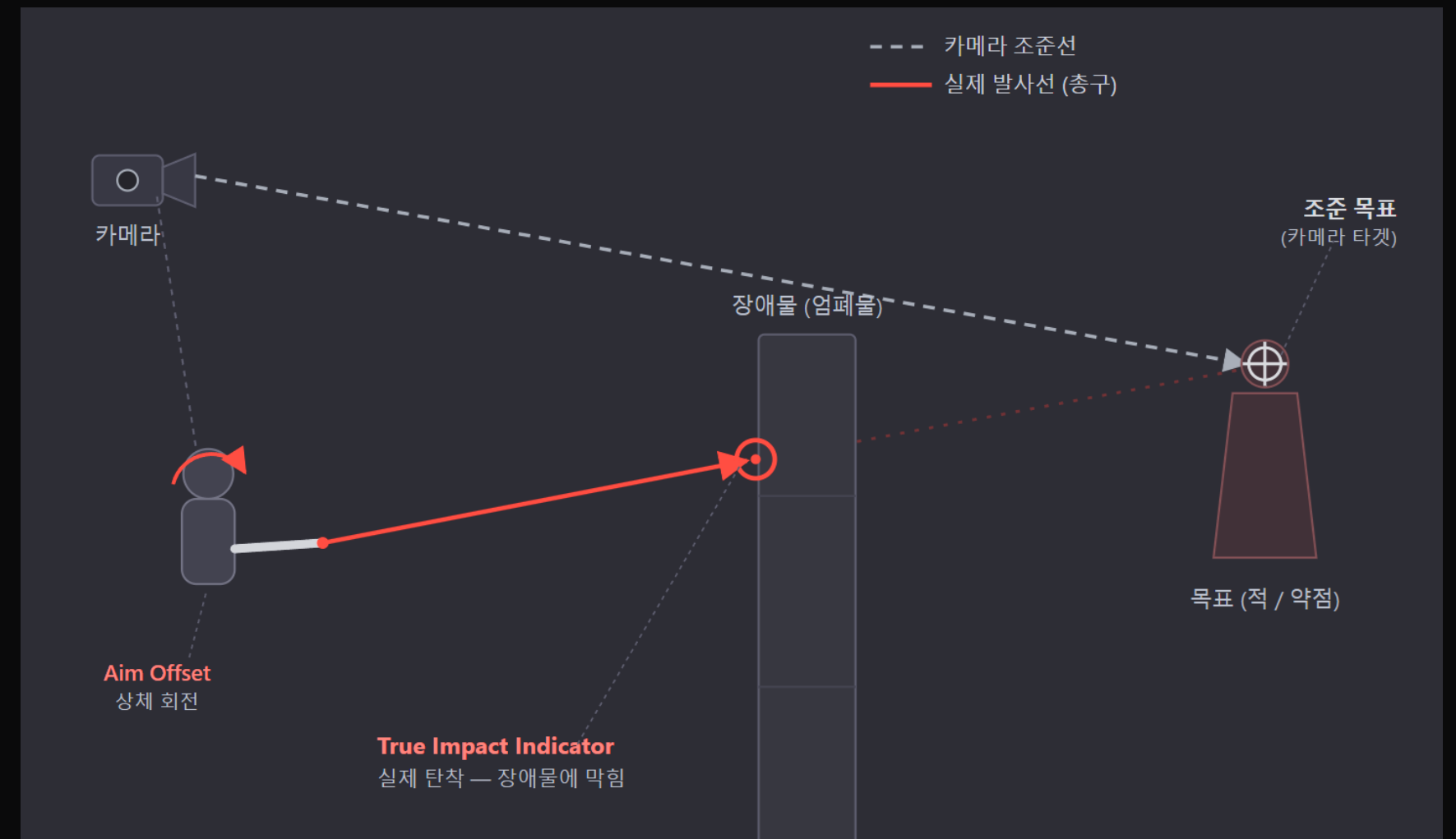


카메라-총구 시차 보정

카메라-총구 시차(Parallax) 보정 — Aim Offset을 활용한 캐릭터 상체 정렬

해결 방법: 총구가 카메라 타겟을 향하도록 상체를 회전시켜 정렬

- 화면 중앙 조준점에서 전방을 향해 Raycast 해서 **카메라 타겟을 결정**
- 총구 전방과 카메라 타겟과의 각도 차이를 계산하고 Aim Offset을 이용해 **상체(총구)와 목표가 일직선에 서도록 정렬**
- 총구에서 다시 카메라 타겟을 향해 Raycast 해서 실제 착탄점을 산출하고 **True Impact Indicator**를 통해 착탄점을 표시



카메라-총구 시차 보정 시연 영상

표면별 GCN 선택 로직

표면에 따른 다른 피격 연출 — 코드 복제 없이 데이터로 정의 및 클라이언트 동기화

문제

- 총기를 사용하는 게임은 사격시 **명중한 표면에서 피드백이 재생되어야 함**
- 표면 피드백이 없으면 맞았는지/어디에 맞았는지 알 수 없음
- 하지만 게임에 쓰이는 표면은 돌/금속/살점 등 매우 많음
- 각 무기와 표면마다 GCN 선택 로직을 작성하면? **무기 N x 표면 $M = N * M$**
- **각 분기마다 코드 추가 필요**

해결 방법

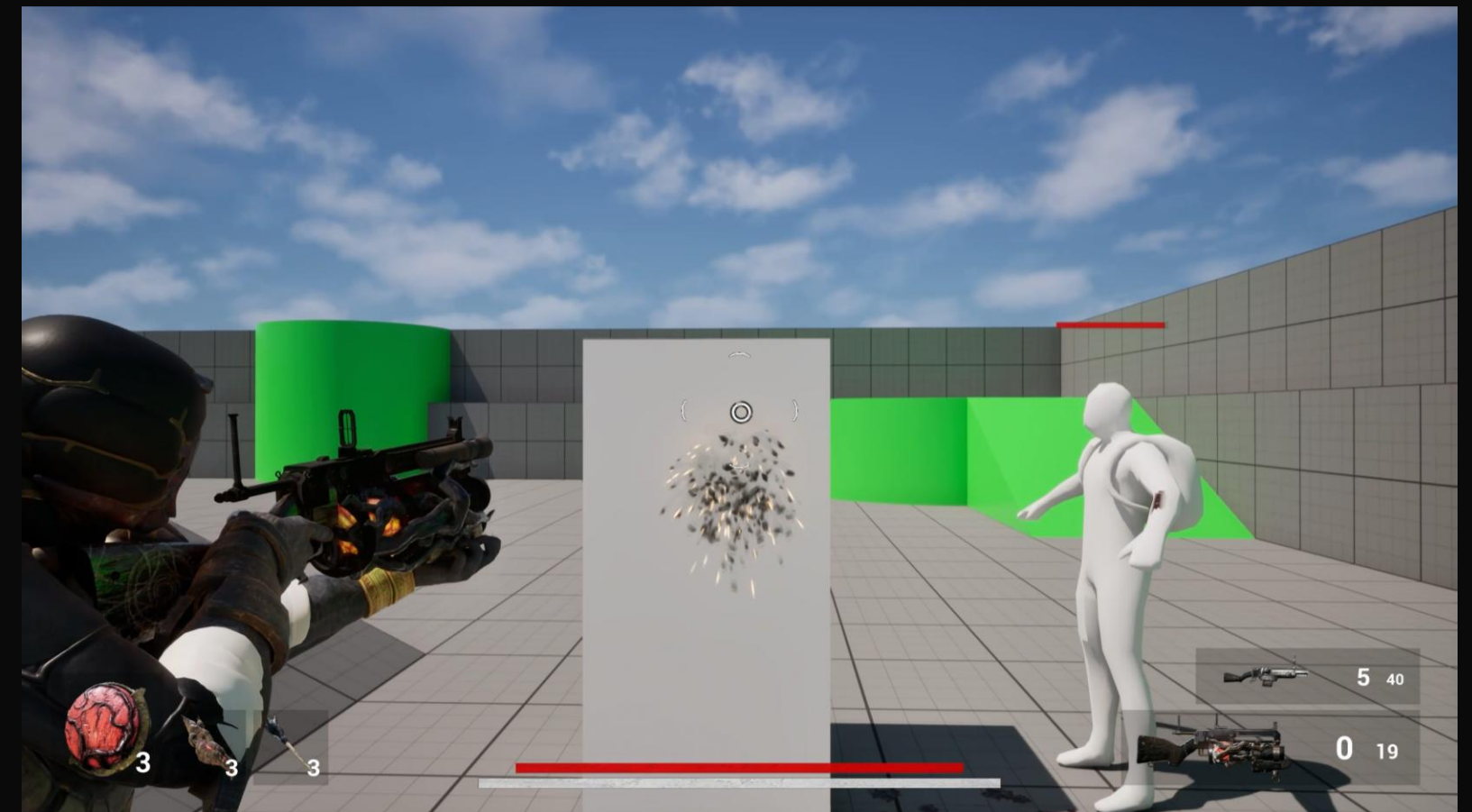
“이 물체는 어떤 표면인가?” 와
“이 무기는 그 표면에 어떤 연출을 쓰나?” 를
분리해서 접근해야 함

표면별 GCN 선택 로직

표면에 따른 다른 피격 연출 — 코드 복제 없이 데이터로 정의 및 클라이언트 동기화

HitResult의 물리 재질을 공통 표면 태그로 바꾸고
→ 그 태그를 기반으로 무기별 피격 GCN을 선택해 재생

- 먼저 피격된 물체의 `FHitResult.PhysMaterial`에서 `SurfaceType`을 읽어
`Surface.\*` 태그로 변환
- 무기 데이터의 `FBlackoutWeaponCueSet.SurfaceImpactCues`에서 해당 표면
태그에 맞는 Impact Cue 태그를 찾음
- 서버 ASC가 해당 태그를 `ExecuteGameplayCue`로 실행 → 다른 클라이언트에
자동 복제되어 모든 클라이언트가 같은 연출을 볼 수 있음
- 표면, 무기 조합이 늘어나도 **데이터만 추가**하면 되도록 설계
- 다른 플레이어에게 반드시 보여야 하는 연출이므로 **서버 권위 실행(+ 로컬 예측 실행)**으로 동기화를 보장



표면별 GCN 시연 영상

서버 권위 및 네트워크 지연 보정

몽타주 + 입력 동기화 — 클라 예측 실행 / 서버 권위 판정

문제

- 근접 콤보·연속 회피 같은 조작은 입력 즉시 캐릭터가 반응해야 조작감이 살아남
- 히트 판정·무적 프레임·콤보 단계 전환 같은 게임 상태는 서버가 확정해야 클라이언트의 치팅 행위, 혹은 클라이언트 간 동기화가 붕괴되는 것을 막을 수 있음
- 하지만 서버 권위를 그대로 우선하면 모든 동작이 클라이언트-서버 왕복 지연(핑)만큼 늦어짐
- **즉각적인 반응성과 서버 권위(공정성)는 정면으로 충돌하는 요구**

해결 방법

- 조작에 따른 예측 실행과 실제 판정을 하는 권위의 역할을 분리
- 로컬 클라이언트는 **서버 응답을 기다리지 않고 동작을 예측 실행**해 핑과 무관한 즉각적 조작감을 확보
- 입력을 서버로 보내 콤보 윈도우·히트 **판정을 서버 권위로 확정**한 뒤 **그 결과를 모든 클라이언트에 동기화**
- **입력 버퍼·핑 적응형 grace·위치 보정 같은 지연 보정 장치**를 더해 예측과 서버 진실의 어긋남을 흡수
- **반응성과 공정성을 동시에 달성할 수 있음**

서버 권위 및 네트워크 지연 보정

몽타주 + 입력 동기화 — 클라 예측 실행 / 서버 권위 판정

입력에 따른 로컬 예측 및 서버 권위 확정 + 동기화 흐름도



근접 콤보와 연속 구르기는 로컬 예측 → 서버 권위 → 원격 동기화로 이어지는 3단계 모델로 동작

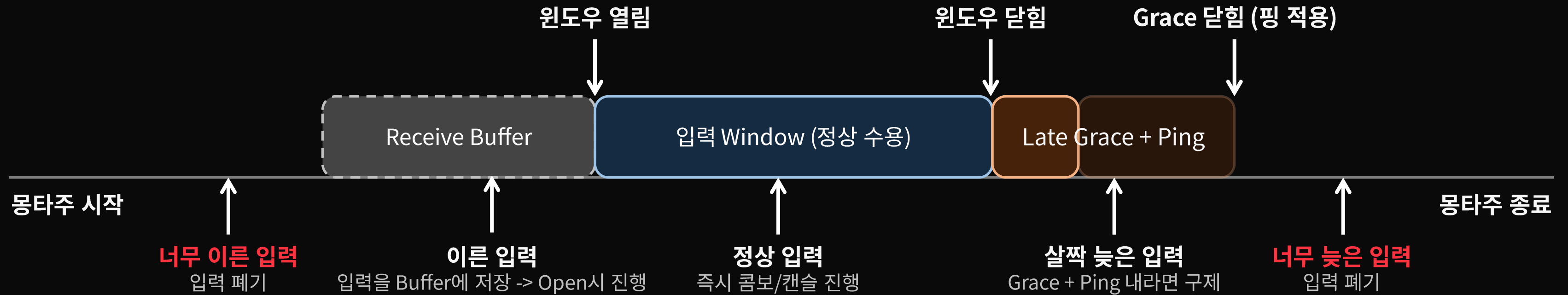
- 입력이 들어오면 **InputSyncPayload**(시퀀스·추정 서버시각·조준 방향 등 동기화용 메타데이터)를 생성하고 곧바로 몽타주를 로컬 예측 실행
- 서버는 복제된 입력을 무조건 신뢰하지 않고 수신 시각과 버퍼 기반으로 콤보 윈도우를 판정한 뒤 다음 단계 진행 여부를 서버 권위로 확정
- 서버가 확정된 결과는 원격 클라이언트에 자동으로 복제되며 입력을 보낸 로컬 클라이언트에게도 되돌아와 미리 실행해 둔 예측을 서버 권위에 맞춰 확정하거나 되돌림.

결과적으로 반응성(예측)과 공정성(서버 권위)을 동시에 달성

서버 권위 및 네트워크 지연 보정

몽타주 + 입력 동기화 — 클라 예측 실행 / 서버 권위 판정

한 콤보/체인 섹션의 입력 타임라인 (서버 월드 타임 기준)



$\text{Grace} = \text{Base} + \text{Ping} * \text{Ping Scale} + \text{Jitter}$

예시) $0.05 + \text{Ping} * 0.5 + 0.04 \rightarrow \text{핑 } 100\text{ms} = 0.14\text{s Grace}$

모든 콤보·체인 입력은 클라이언트 로컬 시간이 아니라 서버 월드 타임이라는 공통 타이머 위에서 판정

- 각 섹션이 시작되면 데이터 자산에 정의된 시각에 맞춰 입력 윈도우가 열리고 이 윈도우 안에 들어온 입력은 정상 수용
- 윈도우가 열리기 전에 도착한 입력은 버퍼에 잠시 보관했다가 윈도우가 열리는 순간 흡수, 윈도우가 닫힌 직후의 살짝 늦은 입력은 late grace 구간으로 구제
- 다만 윈도우·버퍼·grace를 모두 벗어난, 비정상적으로 과거이거나 미래인 입력은 검증 단계에서 폐기

합법적 지연은 관대하게 보정하되 악용 가능한 입력은 엄격히 차단함으로써, 반응성과 서버 권위의 공정성을 동시에 확보

04

프로젝트 회고

개선 점 & 느낀 점

개선점 & 느낀 점

개선 · 보완할 점

- 프로젝트 초기에 다소 미흡했던 **일정 관리**
- AI 에이전트를 활용한 **문서 작업 자동화** 방법론 학습 및 적용
- 더 많은 콘텐츠 (보스전 외의 필드전 등) 구현

느낀 점 / 성과

- 기획 → 구현 → 테스트 → 폴리싱 **게임 개발 전 과정 경험**을 통한 **클라이언트 개발 직무 전반 이해**
- 현업자 멘토링을 받으며 **현업자 시각과 업계 방향성** 체득

“한정된 에셋과 정해진 일정이라는 압박 속에서 팀장으로서 팀원들을 이끌어 프로젝트를 완성시키는 것은 정말 어려운 일이었습니다. 원래 기획했던 목표를 완전히 달성하지 못한 점은 상당히 아쉬웠지만 매우 많은 것을 배울 수 있었던 귀중한 시간이었습니다.”

CLOSING

소울라이크 TPS 위에서 “시키지 않아도 협동하게 되는” 재미를 목표로

언리얼 엔진 GAS · 네트워크 · AI · UI 전 영역을 아우른 완성형 멀티플레이 게임 구현 경험

감사합니다.